

Topic: Maximum flow

What is push relabel algorithm?

Also known as preflow push algorithm.

It is used for computing maximum flow in a flow network.

It is more efficient than Ford-Fulkerson algorithm. Three main operations:

- Preflow()
- Push()
- Relabel()

Constraints The following constraints determine the maximum possible flow from s to t .

o The capacity of the given edge must be greater than the flow of the edge.

o Except for s and t , the incoming and outgoing flow of every edge in the graph should be equal.

⇒ $O(V^2E)$ is time complexity.

⇒ it works in more localized manner than the Ford-Fulkerson method. Rather than examine the entire residual network to find an augmented path, push relabel algorithms work on one vertex at a time, looking only at the vertex's neighbours in the residual network.

⇒ Does not maintain flow conservation property.

Difference between FFA & PRA:

⇒ In FFA the zero value is maintained for the net difference between the total inflow & outflow of every vertex except s & t .
⇒ In PRA the value of inflow to s exceeds then the outflow until value

Ch#1 (Maximum flow)

Network flow:-

A flow network is defined as a directed graph involving a source and sink and several other nodes connected by edges.

In network flow we calculate maximum amount of flow that the network would allow to flow from source to sink.

Ford-Fulkerson Algorithm

Ford-Fulkerson algorithm is considered as one of the most famous algorithms to calculate maximum flow in the network.

It is a greedy approach algorithm.

Rule: Source has only outward edges.

Sink: Sink has only inward edges.

Bottleneck capacity: minimum capacity of an edge in selected path.

Bottleneck capacity of every arbitrary path is considered as flow.

Residual capacity: $\text{Residual capacity of an edge} = \text{Edge capacity} - \text{Flow}$

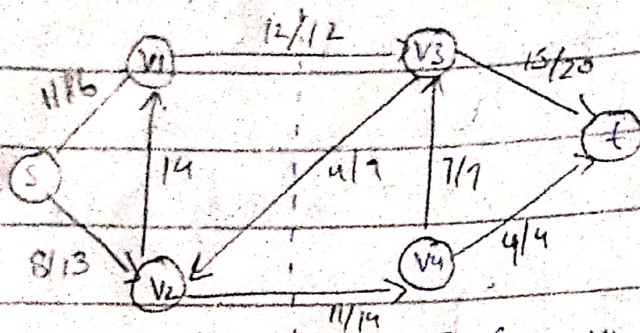
Augmenting path: A path constructed by repeatedly finding a path of free capacity from source to sink & then adding it to the flow. Does not include reverse edges.

Topic (ford-fulkerson)

Cut of flow networks:-

$\Rightarrow$ In order to know whether the flow is maximum or not we will ensure that there is no augmented path in the residual network.

$\Rightarrow$ A cut (S, T) of flow network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$.



$$S = \{v_1, v_2, s\}, \quad T = \{v_3, v_4, t\}$$

$\Rightarrow$ The net flow across (S, T) is $f(S, T) = 19$ and the capacity $c(S, T) = 26$.

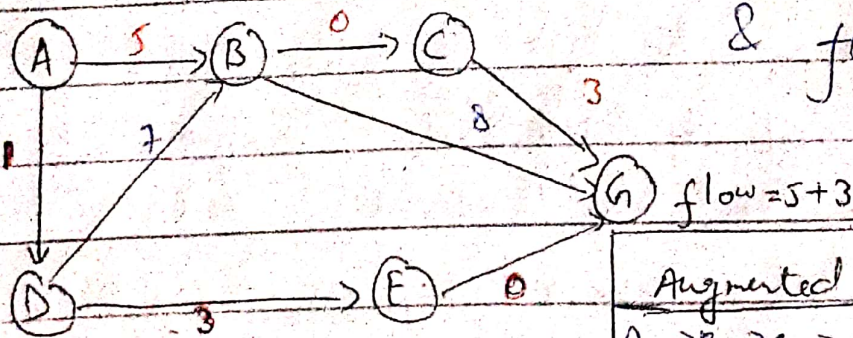
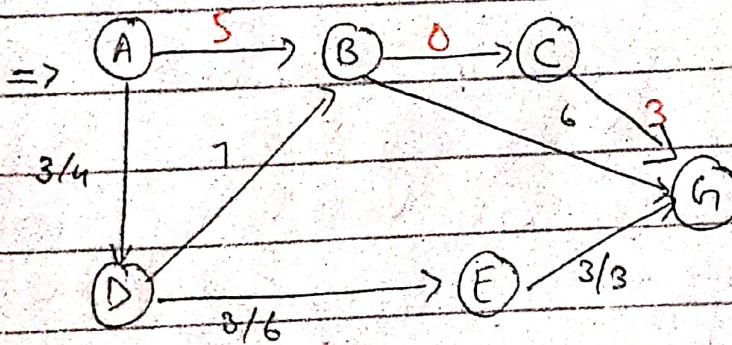
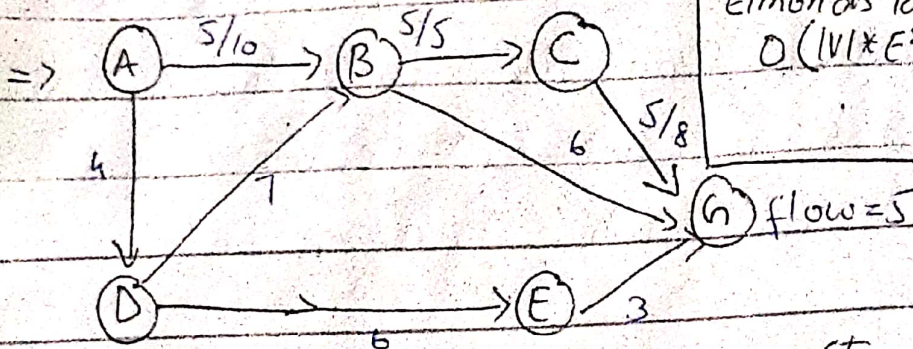
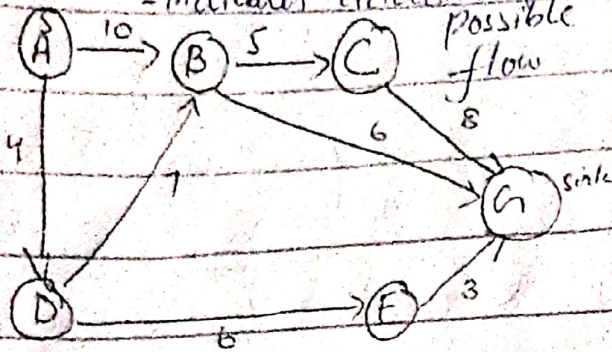
Min cut = capacity of cut = Max flow

Max-flow min-cut theorem:-

$\Rightarrow$ Network flow theorem states that:

"Maximum flow through any network from a given source to a given sink is exactly the sum of the edge weights, if removed, would totally disconnect the source from the sink."

Residual graph indicates how much more flow is allowed in each edge in the network graph.
 → if there is no augmenting path from s to t then flow is maximum - indicates additional possible flow



Maximum flow = 8

Note: if an edge capacity is zero then we cannot choose that edge

FORD-Fulkerson-Method

- 1) Initialize flow $\{0\}$
- 2) while there exist augmenting path p in the residual network $\{f\}$
 - augment flow f along p

Return f

Time complexity:

$= O(|E| * f)$ → maximum flow

Edmonds Karp algorithm
 $O(|V| * E^2)$

Steps

- 1) choose augmented path from source to sink.
- 2) Compute bottleneck capacity.
- 3) Residual capacity & flow.

Augmented path	Bottleneck
A → B → C → G	5
A → D → E → G	3

Topic: "Maximum flow"

that is relabel-to-front algorithm

The relabel-to-front algorithm is more efficient than generic push-relabel.

In push-relabel method, we can apply basic operations of push and relabel on any vertex.

The relabel-to-front algorithm chooses the vertex carefully and manages the network data structures efficiently.

Ford-Fulkerson Push-relabel

keeps a valid flow at all time and improves it until there does

not exist an augmenting path any more.

Net difference b/w total outflow and total inflow for every vertex (excluding s & t) is maintained 0.

$O(V \times f)$

There does not exist an augmenting path at any time and we will

improve the preflow until it is a valid flow.

Allows inflow to exceed the outflow before reaching the final flow.

$O(V^2 E)$

Topic # Task-Scheduling Problem

It is the problem of optimally scheduling a time task on a single processor where each task has a deadline along with a penalty paid.

Unit-time task is a job, such as program to be run on a computer that requires exactly one unit of time to complete.

Task scheduling problem can be modeled as a weighted directed acyclic graph.

2. Select job sequencing for more profit.

The problem of scheduling unit time tasks with deadlines and penalties for a single processor has the following inputs:

- 0 a set $S = \{a_1, a_2, \dots, a_n\}$ of n unit time tasks;

- 0 a set of n integer deadlines $d_1, d_2, \dots, d_n$, such that each d_i satisfies $1 \leq d_i \leq n$

- 0 a set of n non-negative weights or penalties $w_1, w_2, \dots, w_n$.

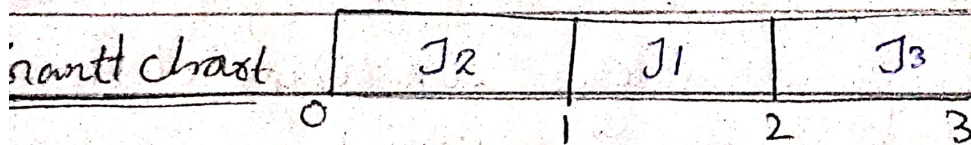
Algorithms-

- 1) Arrange all jobs in decreasing order of profit ($n \log n$)
 - 2) For each job (m_i), do linear search to find particular slot in array of slots where $n = \text{maximum deadline}$, $m = \text{total jobs}$
($n+m$) or n^2
- eg

Job	J ₁	J ₂	J ₃	J ₄	J ₅
Deadline	2	1	3	2	1
Profit	30	50	10	20	10

Arranging job in decreasing order:-

Job	J ₃	J ₁	J ₄	J ₂	J ₅
Deadline	3	2	2	1	1
Profit	10	30	20	50	10



maximum profit = $50 + 30 + 10 = 90$

Time Complexity:-

$O(n^2)$